

DAILY EXPENSE TRACKER : BUDGETBUDDY

Presented By Group 1

TANYA 24/45010

ASHISH MISHRA 24/45021

MANALI TOMAR 24/45090

VAISHNAVI SAXENA 24/45097

Course : Bsc Physical Science with Cs

Semester : 3rd

Subject : Object Oriented Programming With
Python [DSE]

Under the guidance of : Ms Rashmi ma'am

OBJECTIVES

The objective of this project, BudgetBuddy, is to design and develop a Python-based daily expense tracker that enables people to efficiently record, analyze, and manage their day-to-day spending habits for better financial planning and awareness.

The application provides a Graphical User Interface (GUI) built using Tkinter, allowing users to add, view, and analyze expenses in a simple and interactive way by securely storing data in CSV files and generating visual reports to promote responsible financial behavior.

INTRODUCTION

The idea for our project came from our own daily life. Since we live away from our families, we often have to manage our monthly budgets ourselves for food, travel, and other small expenses. So, we decided to create something that's actually useful for us a Daily Expense Tracker that helps record, manage, and analyze our spending easily.

Our project is built in Python and includes a simple graphical user interface (GUI), which allows users to add, view, and calculate their expenses. It mainly uses Python concepts like loops, conditional statements, functions, classes and file handling. This project helped us connect programming with a real-life need and made expense management simpler and more systematic.

TOOLS USED

Hardware Requirement:

Laptop with

Intel core

With at least 2 GB RAM

10 MB free space on Hard Disk

LCD

Operating System & Compiler:

MS Windows

Python IDLE 3.8

Libraries Required:

tkinter, csv, os, datetime, matplotlib

REFERENCES USED

Class Notes

Starting out Python with Tony Gaddis book

METHODOLOGY

Step 1: Importing Libraries:

The required Python libraries such as Tkinter (GUI), CSV (data storage), and Datetime (date of expenses) were imported at the beginning of the program.

Step 2: Creating the GUI Window & Designing Functions:

A main application window was created using Tkinter. Different labels, entry boxes, and buttons were added for user input and actions like Add Expense, View Expense, Delete, Change Password, and Exit.

Step 3: File Handling:

All expense records were stored in a CSV file so that data remains saved even after the program is closed.

Step 4: Data Analysis and Summary:

The program calculates total expenses, displays category-wise data, and can later be extended to show graphs and summaries.

Step 5: Testing the Application:

The project was tested with different inputs to ensure all buttons, functions, and file operations worked correctly.

CODE

```
import tkinter as tk
from tkinter import ttk, messagebox, simpledialog
from datetime import datetime
import csv, os
import matplotlib.pyplot as plt

# File Setup
EXPENSE_FILE = "expenses.csv"
SETTINGS_FILE = "settings.txt"

if not os.path.exists(EXPENSE_FILE):
    with open(EXPENSE_FILE, "w", newline="") as f:
        csv.writer(f).writerow(["Date", "Amount", "Category", "Note"])

if not os.path.exists(SETTINGS_FILE):
    with open(SETTINGS_FILE, "w") as f:
        f.write("password=\n")
        f.write("budget=0\n")
```

```

# Helper Functions
def read_settings():
    with open(SETTINGS_FILE) as f:
        lines = f.readlines()
    data = {}
    for line in lines:
        if "=" in line:
            k, v = line.strip().split("=", 1)
            data[k] = v
    return data

def write_settings(data):
    with open(SETTINGS_FILE, "w") as f:
        for k, v in data.items():
            f.write(f"{k}={v}\n")

# Expense Tracker Class
class ExpenseTracker:
    def __init__(self, root):
        self.root = root
        self.root.title("👛 BudgetBuddy - Personal Expense Tracker")
        self.root.geometry("1200x1200")
        self.root.config(bg="#728FCE")

        self.settings = read_settings()
        self.password_check()

```

```

# Password Protection
def password_check(self):
    if self.settings.get("password") == "":
        new_pass = simpledialog.askstring("Create Password", "Set a new password:")
        if new_pass:
            self.settings["password"] = new_pass
            write_settings(self.settings)
            messagebox.showinfo("Success", "Password created successfully!")
    else:
        entered = simpledialog.askstring("Login", "Enter your password:", show="*")
        if entered != self.settings["password"]:
            messagebox.showerror("Error", "Incorrect password! Exiting...")
            self.root.destroy()
        return
    self.build_ui()

# UI Layout
def build_ui(self):
    tk.Label(self.root, text="💎 BudgetBuddy - Personal Expense Tracker 💎",
            bg="#79BAEC", fg="white", font=("Arial", 20, "bold")).pack(pady=10)

    # Button Frame
    btn_frame = tk.Frame(self.root, bg="#1E90FF")
    btn_frame.pack(pady=5)

```



```

buttons = [
    ("✚ Add Expense", self.add_expense_popup),
    ("📄 View All Expenses", self.view_expenses),
    ("🔍 Search Expenses", self.search_expenses),
    ("▼ Sort Expenses", self.sort_expenses),
    ("📅 Monthly Summary", self.monthly_summary),
    ("💰 Set Budget Limit", self.set_budget),
    ("🔑 Change Password", self.change_password),
    ("🗑 Delete Expense", self.delete_expense),
    ("📊 Graphical Report", self.graph_report),
    ("🚪 Exit", self.exit_app)

]

for i, (txt, cmd) in enumerate(buttons):
    tk.Button(btn_frame, text=txt, command=cmd, bg="purple", font=("Arial", 11, "bold"), width=18)\
        .grid(row=i//2, column=i%2, padx=10, pady=5)

# Table
self.columns = ("Date", "Amount", "Category", "Note")
self.table = ttk.Treeview(self.root, columns=self.columns, show="headings", height=15)
for col in self.columns:
    self.table.heading(col, text=col)
    self.table.column(col, width=180)
self.table.pack(pady=10)
self.scroll = ttk.Scrollbar(self.root, orient="vertical", command=self.table.yview)
self.table.configure(yscroll=self.scroll.set)
self.scroll.pack(side="right", fill="y")

```

```
# Total Label
self.total_label = tk.Label(self.root, text="", bg="#1E90FF", fg="white", font=("Arial", 14, "bold"))
self.total_label.pack(pady=10)
self.view_expenses()

# Add Expense
def add_expense_popup(self):
    win = tk.Toplevel(self.root)
    win.title("Add Expense")
    win.geometry("400x300")
    win.config(bg="#1E90FF")

    labels = ["Amount (₹):", "Category:", "Note:"]
    entries = []
    for i, lbl in enumerate(labels):
        tk.Label(win, text=lbl, bg="#1E90FF", fg="white", font=("Arial", 12, "bold")).grid(row=i, column=0, padx=10, pady=10)
        e = tk.Entry(win, width=25)
        e.grid(row=i, column=1)
        entries.append(e)

    def save():
        try:
            amt = float(entries[0].get())
            cat = entries[1].get()
            note = entries[2].get()
            date = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
            with open(EXPENSE_FILE, "a", newline="") as f:
                csv.writer(f).writerow([date, amt, cat, note])
            messagebox.showinfo("Success", "Expense added!")
            win.destroy()
```

```

        self.view_expenses()
        self.check_budget()
    except ValueError:
        messagebox.showerror("Error", "Enter a valid amount!")

tk.Button(win, text="Save", command=save, bg="white", fg="black", font=("Arial", 11, "bold")).grid(row=4, column=1, pady=15)

# View Expenses
def view_expenses(self):
    for i in self.table.get_children():
        self.table.delete(i)
    total = 0
    with open(EXPENSE_FILE) as f:
        reader = csv.reader(f)
        next(reader)
        for row in reader:
            self.table.insert("", "end", values=row)
            total += float(row[1])
    self.total_label.config(text=f"💰 Total Spent: ₹{total:.2f}")

# Search Expenses
def search_expenses(self):
    term = simpledialog.askstring("Search", "Enter keyword/category/date:")
    if not term:
        return
    for i in self.table.get_children():
        self.table.delete(i)
    with open(EXPENSE_FILE) as f:
        reader = csv.reader(f)
        next(reader)

```

```

        for row in reader:
            if any(term.lower() in cell.lower() for cell in row):
                self.table.insert("", "end", values=row)

# Sort Expenses
def sort_expenses(self):
    choice = simpledialog.askstring("Sort", "Sort by (amount/date/category):")
    if not choice:
        return
    data = []
    with open(EXPENSE_FILE) as f:
        reader = csv.reader(f)
        next(reader)
        data = [row for row in reader]
    if choice.lower() == "amount":
        data.sort(key=lambda x: float(x[1]))
    elif choice.lower() == "category":
        data.sort(key=lambda x: x[2].lower())
    elif choice.lower() == "date":
        data.sort(key=lambda x: x[0])
    for i in self.table.get_children():
        self.table.delete(i)
    for row in data:
        self.table.insert("", "end", values=row)

# Monthly Summary
def monthly_summary(self):
    summary = {}
    with open(EXPENSE_FILE) as f:
        reader = csv.reader(f)
        next(reader)
        for row in reader:
            month = row[0][:7]
            summary[month] = summary.get(month, 0) + float(row[1])

```

```
result = "\n".join([f"{m}: ₹{t:.2f}" for m, t in summary.items()])
messagebox.showinfo("Monthly Summary", result or "No expenses yet!")

# Budget
def set_budget(self):
    new_budget = simpledialog.askfloat("Set Budget", "Enter monthly budget (₹):")
    if new_budget is not None:
        self.settings["budget"] = str(new_budget)
        write_settings(self.settings)
        messagebox.showinfo("Success", "Budget set successfully!")

def check_budget(self):
    budget = float(self.settings.get("budget", 0))
    if budget <= 0:
        return
    total = 0
    with open(EXPENSE_FILE) as f:
        reader = csv.reader(f)
        next(reader)
        for row in reader:
            total += float(row[1])
    if total > budget:
        messagebox.showwarning("Budget Exceeded", f"⚠ You've exceeded your budget of ₹{budget:.2f}!")

# Change Password
def change_password(self):
    old = simpledialog.askstring("Change Password", "Enter old password:", show="*")
    if old != self.settings["password"]:
        messagebox.showerror("Error", "Incorrect password!")
    return
```



```
new = simpdialog.askstring("Change Password", "Enter new password:", show="*")
if new:
    self.settings["password"] = new
    write_settings(self.settings)
    messagebox.showinfo("Success", "Password changed successfully!")

# Delete Expense
def delete_expense(self):
    selected = self.table.selection()
    if not selected:
        messagebox.showerror("Error", "Select an expense to delete!")
        return
    values = self.table.item(selected[0])["values"]
    self.table.delete(selected[0])

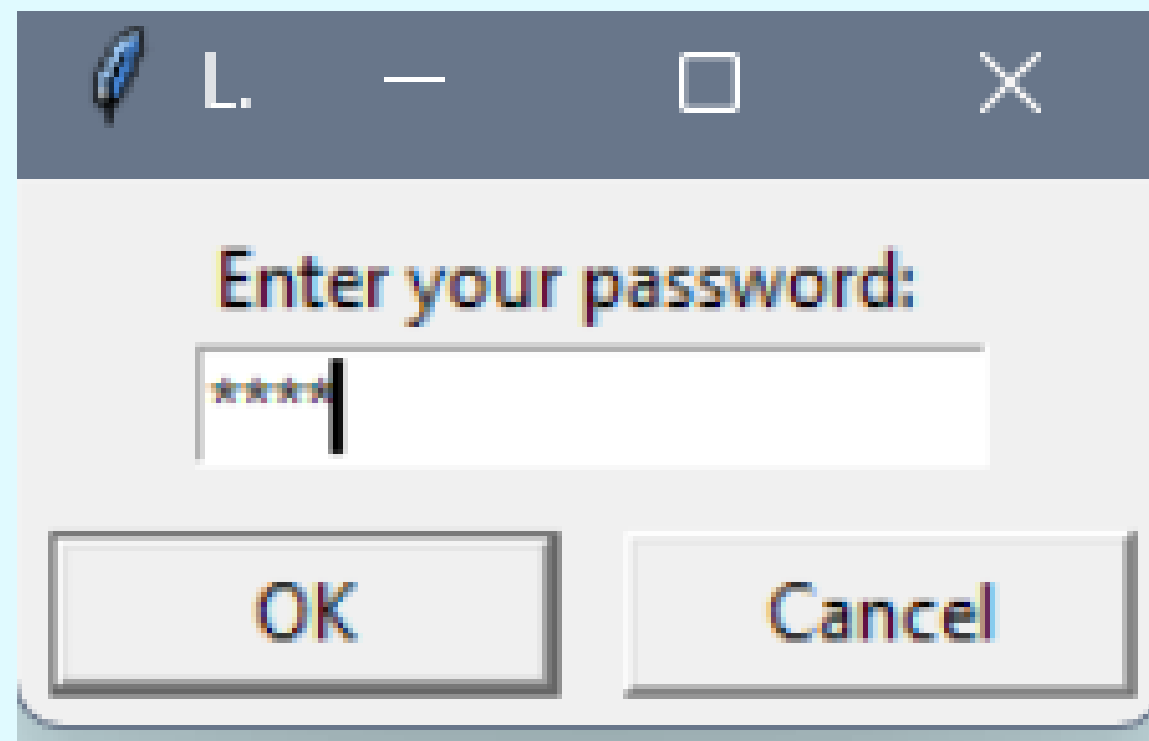
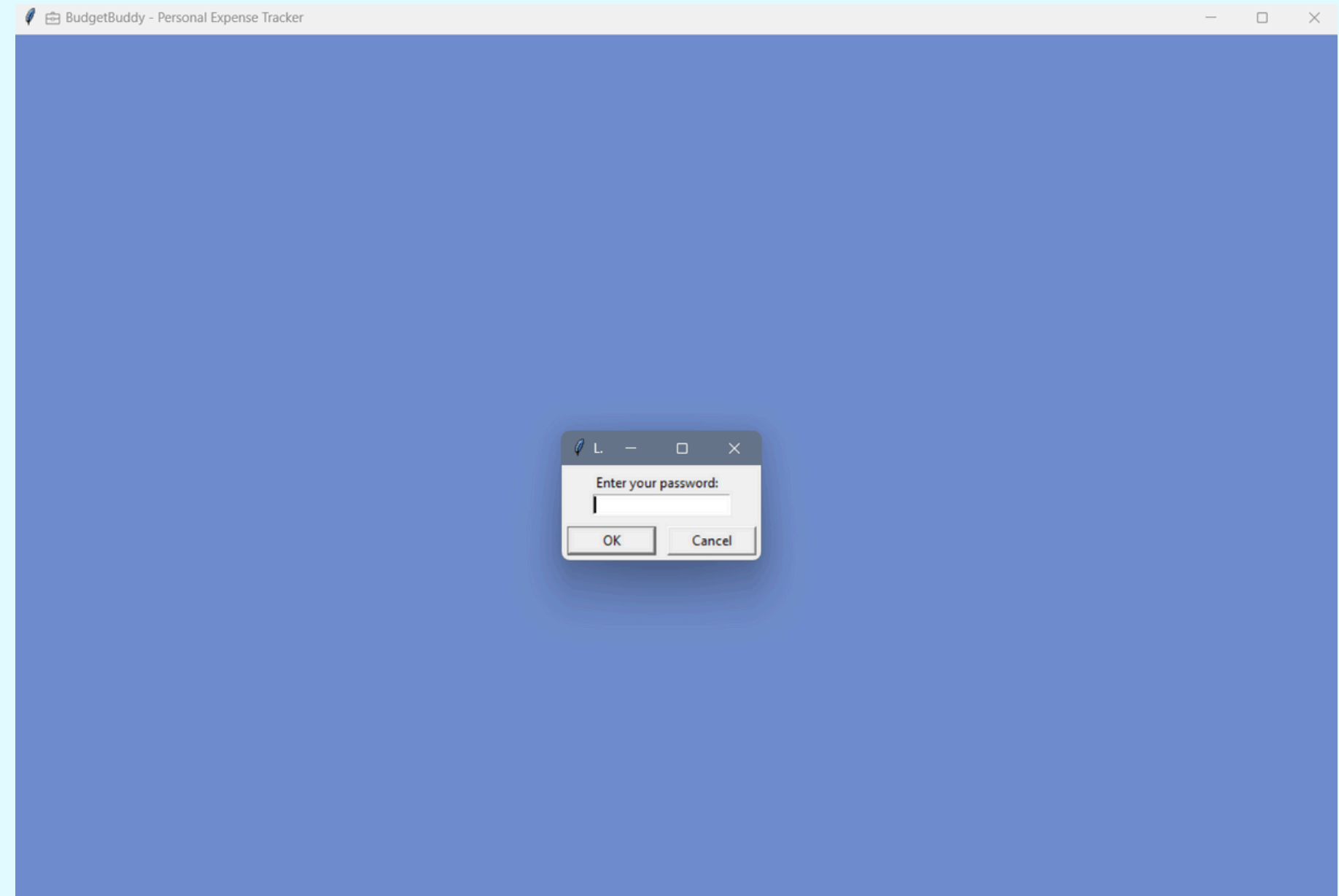
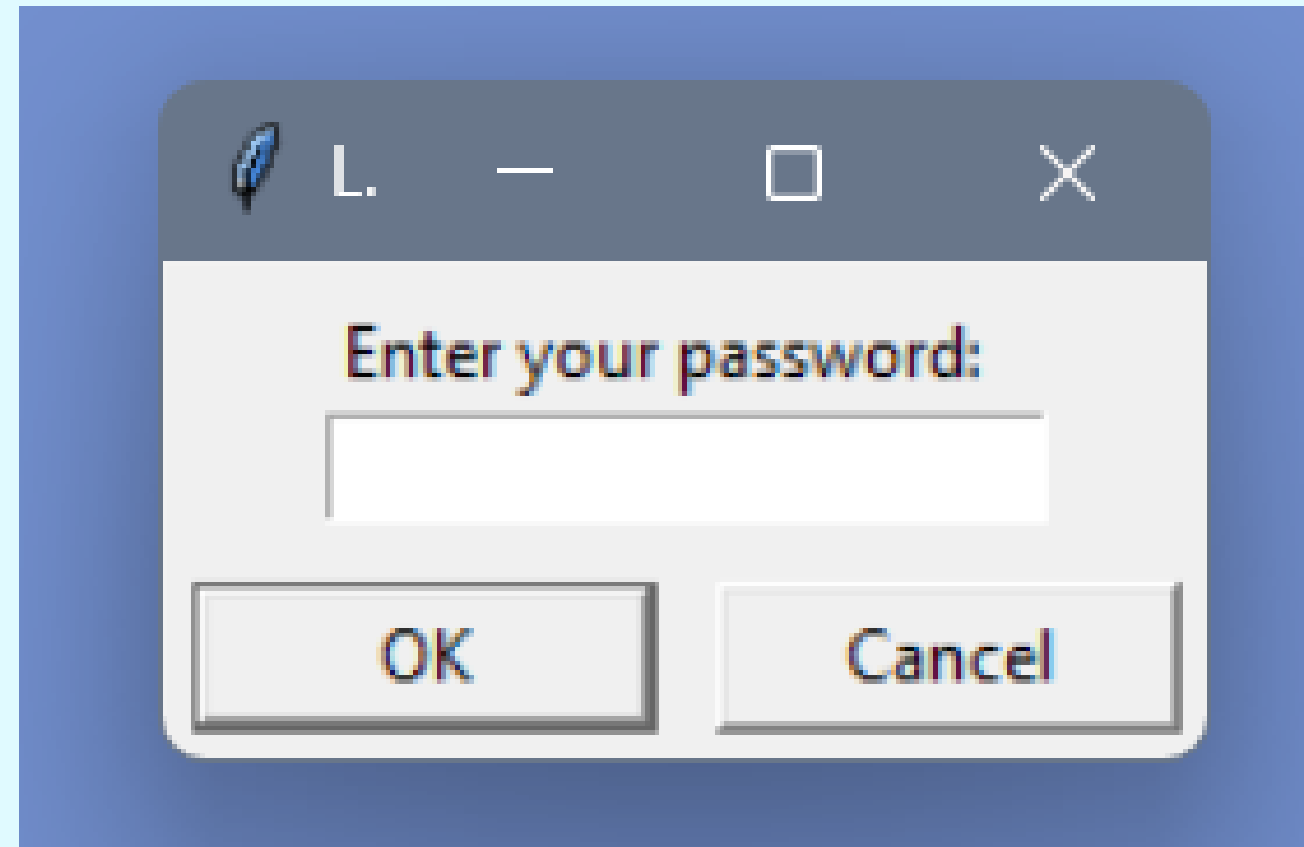
    with open(EXPENSE_FILE) as f:
        lines = list(csv.reader(f))
    with open(EXPENSE_FILE, "w", newline="") as f:
        writer = csv.writer(f)
        for row in lines:
            if row != values:
                writer.writerow(row)
    messagebox.showinfo("Deleted", "Expense deleted successfully!")
```

```
# Graphical Report
def graph_report(self):
    data = {}
    with open(EXPENSE_FILE) as f:
        reader = csv.reader(f)
        next(reader)
        for row in reader:
            cat = row[2]
            data[cat] = data.get(cat, 0) + float(row[1])
    if not data:
        messagebox.showinfo("Info", "No data to plot!")
        return
    plt.bar(data.keys(), data.values(), color="skyblue")
    plt.title("BudgetBuddy - Expense by Category")
    plt.xlabel("Category")
    plt.ylabel("Total Spent (₹)")
    plt.show()

# Exit App
def exit_app(self):
    confirm = messagebox.askyesno("Exit", "Are you sure you want to exit BudgetBuddy?")
    if confirm:
        self.root.destroy()

# Run App
root = tk.Tk()
ExpenseTracker(root)
root.mainloop()
```

OUTPUT



BudgetBuddy – Personal Expense Tracker

+ Add Expense

View All Expenses

Search Expenses

Sort Expenses

Monthly Summary

Set Budget Limit

Change Password

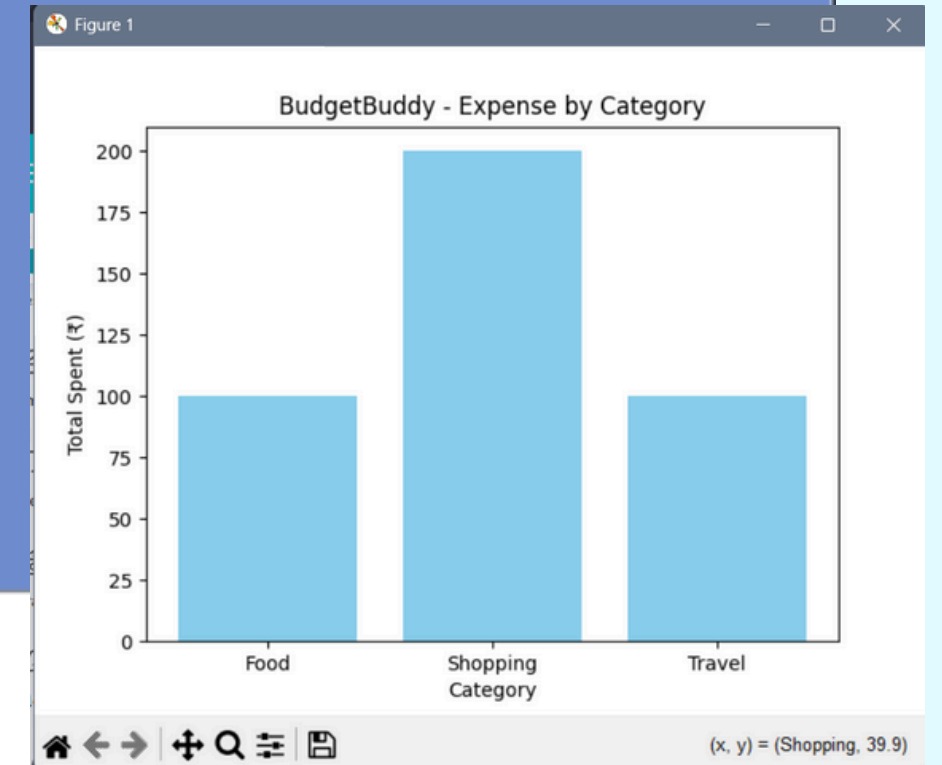
Delete Expense

Graphical Report

Exit

Date	Amount	Category	Note
2025-11-03 20:12:37	100.0	Food	expensive!
2025-11-03 20:13:21	200.0	Shopping	had fun!
2025-11-03 20:40:56	100.0	Travel	

Total Spent: ₹400.00



Monthly Summary



2025-11: ₹400.00

OK

ADVANTAGES

- User-friendly interface, easy to modify as well.
- Stores data persistently in files, i.e, data permanence.
- Helps maintain financial discipline by simple expense tracking.
- Graphical analysis enhances understanding.
- Secure with password protection.

DISADVANTAGES

- Case sensitive functions , i.e, if user enters two different formats for the same activity (food or Food) , then it will be counted as two different entities.
- Limited to single-user access
- Manual data entry required
- No cloud backup or automatic syncing yet

CONCLUSION

BudgetBuddy is a simple yet effective Python project that helps users keep track of their daily expenses through an easy-to-use GUI.

It makes managing money easier by allowing users to record, view, and analyze their spending habits.

The project not only improves financial awareness but also shows how Python and Tkinter can be used to create practical, real-life applications.

THANK YOU!